Object oriented Programming with C++ LAB

Program 1

Develop a c++ program to find the largest of three numbers .

```
#include <iostream.h>
#include <conio.h>
void main() {
   int a, b, c;
   clrscr();  // clear screen
   cout << "Enter three numbers: ";
   cin >> a >> b >> c;
   if (a >= b && a >= c)
      cout << "\nLargest number is: " << a;
   else if (b >= a && b >= c)
      cout << "\nLargest number is: " << b;
   else
      cout << "\nLargest number is: " << c;
  getch();  // wait for key press
}
```

Program 2

Develop a c++ program to sort the elements in ascending and descending order

```
#include <iostream.h>
#include <conio.h>
void main() {
    int arr[50], n, i, j, temp;
  clrscr();  // clear screen
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "\nEnter " << n << " elements:\n";
    for (i = 0; i < n; i++)
        cin >> arr[i];
// Sort in ascending order
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (arr[i] > arr[j]) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
cout << "\nElements in Ascending Order:\n";
    for (i = 0; i < n; i++)
```

```
        cout << arr[i] << " ";

    // Display in descending order (reverse)
    cout << "\n\nElements in Descending Order:\n";
    for (i = n - 1; i >= 0; i--)
        cout << arr[i] << " ";

    getch();  // wait for key press
}
```

Program 3

Develop a c++ program using classes to display student name roll number marks obtained in 2 subjects and total score of students.

```
#include <iostream.h>

#include <conio.h>

class Student {

private:

   char name[50];

   int roll;

   float mark1, mark2, total;

public:

   void getData() {

      cout << "Enter student name: ";

      cin >> name;


      cout << "Enter roll number: ";

      cin >> roll;


      cout << "Enter marks in subject 1: ";

      cin >> mark1;


      cout << "Enter marks in subject 2: ";

      cin >> mark2;


      total = mark1 + mark2;
```

```
    }

    void displayData() {
        cout << "\n--- Student Details ---";
        cout << "\nName: " << name;
        cout << "\nRoll Number: " << roll;
        cout << "\nMarks in Subject 1: " << mark1;
        cout << "\nMarks in Subject 2: " << mark2;
        cout << "\nTotal Marks: " << total;
    }
};

void main() {
    clrscr();   // clear the screen

    Student s;  // create object
    s.getData();
    s.displayData();

    getch();    // wait for key press
}
```

Program 4

Develop c++ program, for bank employee to print name of the employee account_no and balance print invalid balance if amount is less than 500 display the same also display the balance after withdraw and deposit.

```
#include <iostream.h>

#include <conio.h>

#include <string.h>


class BankEmployee {

private:

    char name[50];

    long accountNumber;

    float balance;


public:

    void getData() {

        cout << "Enter Employee Name: ";

        cin >> name;


        cout << "Enter Account Number: ";

        cin >> accountNumber;


        cout << "Enter Initial Balance: ";

        cin >> balance;
```

```
    if (balance < 500) {
        cout << "\nInvalid Balance! Minimum balance should be 500.\n";
    }
}

void deposit() {
    float amount;
    cout << "\nEnter Amount to Deposit: ";
    cin >> amount;
    balance += amount;
    cout << "Amount Deposited Successfully!\n";
}

void withdraw() {
    float amount;
    cout << "\nEnter Amount to Withdraw: ";
    cin >> amount;

    if (amount > balance) {
        cout << "Insufficient Balance!\n";
    } else {
        balance -= amount;
        cout << "Amount Withdrawn Successfully!\n";
    }
}
```

```
    void display() {
        cout << "\n--- Employee Account Details ---";
        cout << "\nName: " << name;
        cout << "\nAccount Number: " << accountNumber;
        cout << "\nCurrent Balance: " << balance << "\n";
    }
};

void main() {
    clrscr();   // Clear the screen

    BankEmployee emp;

    emp.getData();

    cout << "\nInitial Details:";
    emp.display();

    emp.deposit();
    cout << "\nAfter Deposit:";
    emp.display();

    emp.withdraw();
    cout << "\nAfter Withdrawal:";
    emp.display();
```

```
    getch();    // Wait for key press
}
```

Program 5

Develop c++ program to demonstrate function overloading for the following prototype

add(int a, int b),

add(double a, double b

```
#include <iostream.h>
#include <conio.h>

// Function 1: adds two integers
int add(int a, int b) {
   return a + b;
}

// Function 2: adds two double (floating point) numbers
double add(double a, double b) {
   return a + b;
}

void main() {
   clrscr();   // clear screen

   int x, y;
   double p, q;

   cout << "Enter two integers: ";
   cin >> x >> y;
```

```
    cout << "Enter two decimal numbers: ";
    cin >> p >> q;

    cout << "\nSum of integers = " << add(x, y);
    cout << "\nSum of doubles  = " << add(p, q);

    getch();    // wait for key press
}
```

Program 6

Develop a c++ program using operator overloading for overloading unary minus operator.

```
#include <iostream.h>
#include <conio.h>

class Number {
private:
    int x, y;

public:
    void getData() {
        cout << "Enter two numbers: ";
        cin >> x >> y;
    }

    void display() {
        cout << "x = " << x << ", y = " << y << "\n";
    }

    // Overload unary minus operator
    void operator -() {
        x = -x;
        y = -y;
    }
};
```

```
void main() {
    clrscr();   // clear screen

    Number n;

    n.getData();
    cout << "\nBefore applying unary minus:\n";
    n.display();

    -n;  // calls overloaded operator
    cout << "\nAfter applying unary minus:\n";
    n.display();

    getch();    // wait for key press
}
```

Program 7

Develop c++ program to implement multiple inheritance for performing arithmetic operation of 2 numbers.

```
#include <iostream.h>
#include <conio.h>

class Input {
protected:
    int a, b;
public:
    void getData() {
        cout << "Enter two numbers: ";
        cin >> a >> b;
    }
};

class Operations {
public:
    int add(int x, int y) {
        return x + y;
    }
    int subtract(int x, int y) {
        return x - y;
    }
    int multiply(int x, int y) {
        return x * y;
```

```
    }
    float divide(int x, int y) {
        if (y == 0) {
            cout << "Division by zero not allowed!\n";
            return 0;
        }
        return (float)x / y;
    }
};

// Derived class using multiple inheritance
class Result : public Input, public Operations {
public:
    void displayResults() {
        cout << "\n--- Arithmetic Operations ---";
        cout << "\nAddition: " << add(a, b);
        cout << "\nSubtraction: " << subtract(a, b);
        cout << "\nMultiplication: " << multiply(a, b);
        cout << "\nDivision: " << divide(a, b);
    }
};

void main() {
    clrscr();  // clear screen

    Result r;
```

```
    r.getData();
    r.displayResults();

    getch();   // wait for key press
}
```

Program 8

Develop a c++ program using constructor in derived class s to initialize alpha beta and gamma and display corresponding values.

```
#include <iostream.h>
#include <conio.h>

class Base1 {
protected:
   int alpha;
public:
   Base1(int a) {     // Constructor of Base1
      alpha = a;
   }
};

class Base2 {
protected:
   int beta;
public:
   Base2(int b) {     // Constructor of Base2
      beta = b;
   }
};

class Derived : public Base1, public Base2 {
private:
```

```
    int gamma;
public:
    // Constructor of Derived class initializing all three
    Derived(int a, int b, int c) : Base1(a), Base2(b) {
        gamma = c;
    }

    void display() {
        cout << "\nAlpha = " << alpha;
        cout << "\nBeta  = " << beta;
        cout << "\nGamma = " << gamma;
    }
};

void main() {
    clrscr();   // Clear screen

    Derived obj(10, 20, 30);   // Create object with values
    cout << "--- Values Initialized Using Constructors ---\n";
    obj.display();

    getch();    // Wait for key press
}
```

Program 9

Develop c++ program to create a text file , check file created or not if created it will write some text into the file and then read the text from the file.

```
#include <iostream.h>

#include <fstream.h>

#include <conio.h>


void main() {
   clrscr();   // clear screen


   ofstream fout("sample.txt");   // create file for writing


   if (!fout) {   // check if file created successfully
      cout << "Error: File could not be created!";
   } else {
      cout << "File created successfully!\n";


      fout << "This is a sample text written to the file.\n";
      fout << "Turbo C++ file handling demonstration.\n";


      fout.close();   // close the file after writing
      cout << "Data written to file successfully.\n";
   }


   ifstream fin("sample.txt");   // open file for reading
   if (!fin) {
```

```
        cout << "\nError: File could not be opened for reading!";
    } else {
        cout << "\nReading data from file:\n\n";

        char ch;
        while (fin.get(ch)) {     // read character by character
            cout << ch;
        }
        fin.close();   // close the file
    }

    getch();   // wait for key press
}
```

Program 10

Develop c++ program to write and read time in /from binary file using fstream.

```
#include <iostream.h>
#include <fstream.h>
#include <conio.h>

class Time {
   int hr, min, sec;

public:
   void getTime() {
      cout << "Enter hours: ";
      cin >> hr;
      cout << "Enter minutes: ";
      cin >> min;
      cout << "Enter seconds: ";
      cin >> sec;
   }

   void displayTime() {
      cout << hr << ":" << min << ":" << sec << "\n";
   }
};
```

```
void main() {
   clrscr();  // clear screen

   Time t1, t2;

   cout << "--- Enter Time Details ---\n";
   t1.getTime();

   fstream file;
   file.open("time.dat", ios::out | ios::binary);  // open for writing in binary mode

   if (!file) {
      cout << "Error: File could not be created!";
   } else {
      file.write((char*)&t1, sizeof(t1));  // write object to file
      cout << "\nTime written to file successfully.\n";
      file.close();
   }

   file.open("time.dat", ios::in | ios::binary);  // open for reading

   if (!file) {
      cout << "\nError: File could not be opened for reading!";
   } else {
      file.read((char*)&t2, sizeof(t2));   // read object from file
      cout << "\nTime read from file: ";
```

```
        t2.displayTime();
        file.close();
    }

    getch();  // wait for key press
}
```

Develop a function which throws a division by zero exception and catch it in catch block. Write a C++ program to demonstrate usage of try, catch and throw to handle exception.

```
#include <iostream.h>
#include <conio.h>

void divide(int a, int b)
{
   if (b == 0)
      throw "Division by zero error!";  // throwing an exception
   else
      cout << "Result: " << a / b << endl;
}

void main()
{
   clrscr();  // clear screen (Turbo C specific)

   int x, y;
   cout << "Enter numerator: ";
   cin >> x;
   cout << "Enter denominator: ";
   cin >> y;

   try
   {
```

```
        divide(x, y);
    }
    catch (const char *msg)   // catching the thrown exception
    {
        cout << "Exception caught: " << msg << endl;
    }

    cout << "\nProgram continues normally..." << endl;

    getch();  // wait for key press before closing the output window
}
```

Program 12

Develop a c++ program that handles array out of bound exception .

```
#include <iostream.h>
#include <conio.h>

void main() {
   clrscr();   // clear screen

   int arr[5] = {10, 20, 30, 40, 50};
   int index;

   cout << "Enter array index (0 to 4): ";
   cin >> index;

   try {
      if (index < 0 || index >= 5)
         throw index;   // throw exception if out of range

      cout << "Value at index " << index << " = " << arr[index] << "\n";
   }
   catch (int e) {
      cout << "Exception caught! Invalid array index: " << e << "\n";
   }
```

```
    getch();    // wait for key press
}
```